

Warframe Manager

Skill Development & Project Ramp-Up Plan

This document outlines a structured week-by-week learning and development roadmap for building Warframe Manager using React, Tauri, Python, and SQLite. The focus is on sustainable skill growth, vertical-slice development, and long-term maintainability.

Week 1 — Development Environment & Core Concepts

- ■ Install and configure Node.js, Rust, Python, SQLite tools, and Git.
- ■ Learn basic terminal workflows and project navigation.
- ■ Understand frontend vs backend vs storage architecture.
- ■ Create a minimal React app using Vite.
- ■ Learn JSX, components, props, and state.
- ■ Create and push your first Git repository.

Week 2 — React Fundamentals

- ■ Build reusable React components.
- ■ Learn useState and useEffect.
- ■ Create forms and controlled inputs.
- ■ Build a searchable mock UI.
- ■ Learn basic CSS/Tailwind layout concepts.
- ■ Practice component organization and folder structure.

Week 3 — Python Backend Fundamentals

- ■ Create a standalone Python backend service.
- ■ Learn FastAPI basics.
- ■ Create REST endpoints returning JSON.
- ■ Handle async requests and error responses.
- ■ Organize backend services into modules.
- ■ Create a basic logging system.

Week 4 — Frontend ↔ Backend Communication

- ■ Connect React frontend to Python backend.
- ■ Send API requests using fetch/axios.
- ■ Display returned JSON in React components.

- ■ Implement loading and error states.
- ■ Understand API contracts and serialization.
- ■ Create the first complete data flow.

Week 5 — SQLite & Caching

- ■ Learn SQLite fundamentals.
- ■ Store fetched API responses locally.
- ■ Create simple database schemas.
- ■ Implement cache expiration logic.
- ■ Separate storage logic from API logic.
- ■ Practice reading/writing structured data.

Week 6 — Tauri Integration

- ■ Set up Tauri with the React frontend.
- ■ Learn the Tauri build system.
- ■ Run the app as a native desktop application.
- ■ Understand IPC and desktop packaging.
- ■ Test Windows/Linux compatibility.
- ■ Learn application configuration handling.

Week 7 — First Vertical Slice

- ■ Build the Market Lookup feature.
- ■ Search a Warframe item from the frontend.
- ■ Fetch and cache market data in the backend.
- ■ Display clean results in the UI.
- ■ Add loading/error handling.
- ■ Verify the architecture works end-to-end.

Week 8 — Cleanup & Architecture Stabilization

- ■ Refactor messy code and improve naming.
- ■ Separate services into clean modules.
- ■ Document architecture decisions.
- ■ Create configuration systems.
- ■ Add centralized error handling.
- ■ Review folder structure and scalability.

Week 9 — UI Systems & Tables

- ■ Build reusable tables and cards.
- ■ Implement scrollable data views.
- ■ Learn state management patterns.
- ■ Add filtering/searching systems.
- ■ Improve layout consistency.
- ■ Create reusable UI utilities.

Week 10 — Analytics & Visualization

- ■ Learn charting libraries.
- ■ Render basic market trend graphs.
- ■ Build statistics cards.
- ■ Practice transforming API data for visualization.
- ■ Improve responsiveness and rendering performance.
- ■ Create mock analytics dashboards.

Week 11 — OCR & Image Processing Research

- ■ Experiment with OCR libraries.
- ■ Test static screenshot parsing.
- ■ Evaluate OCR reliability in Warframe UI contexts.
- ■ Prototype inventory/mod screen parsing.
- ■ Document realistic OCR limitations.
- ■ Avoid attempting live combat OCR systems yet.

Week 12 — Project Consolidation

- ■ Review architecture and eliminate technical debt.
- ■ Create milestone goals for Alpha/Beta versions.
- ■ Write onboarding/setup documentation.
- ■ Prepare future roadmap planning.
- ■ Stabilize the first usable release candidate.
- ■ Identify future systems that can be modular plugins.

Key Principle: Your first objective is not to build every feature. Your first objective is to prove that the architecture works. Focus on small, complete vertical slices rather than massive unfinished systems.